



Initiation à Symfony

Un Framework PHP

Sommaire

Sommaire.....	1
Gestion de la Base de Données avec Doctrine.....	2
1. Configuration de Doctrine.....	2
Vérifier la connexion à la base de données.....	2
Vérifier la connexion.....	2
2. Création des Entités	3
3. Génération et Application des Migrations.....	3
4. Manipuler la Base de Données avec Doctrine	4
Ajouter un Vol en Base de Données.....	4
Récupérer les Vols.....	4
Conclusion.....	5



Gestion de la Base de Données avec Doctrine

Objectifs de la séance

Dans cette séance, nous allons :

- ✓ Configurer Doctrine et la connexion à la base de données.
- ✓ Créer des entités basées sur notre modèle de données.
- ✓ Générer et appliquer des migrations.
- ✓ Manipuler la base de données avec Doctrine (CRUD).

1. Configuration de Doctrine

Vérifier la connexion à la base de données

Symfony utilise Doctrine comme ORM (Object Relational Mapper) pour interagir avec la base de données. Commençons par configurer notre connexion.

Ouvrons le fichier .env situé à la racine du projet et modifions la ligne suivante :

```
DATABASE_URL="mysql://root:password@127.0.0.1:3306/gestion_vol?serverVersion=8.0"
```

- Remplacez root par votre utilisateur MySQL.
- Remplacez password par votre mot de passe MySQL.
- gestion_vol est le nom de notre base de données.
- serverVersion=8.0 doit correspondre à votre version MySQL.

Vérifier la connexion

Exécutons la commande suivante pour vérifier que Symfony peut bien se connecter à MySQL :

- php bin/console doctrine:database:create

Cela créera la base de données si elle n'existe pas.



2. Création des Entités

Une **entité** est une classe PHP représentant une table dans la base de données. Créons une entité Vol correspondant à la table vol.

- php bin/console make:entity
- Vol

Cela nous demandera d'ajouter des propriétés. Ajoutons :

- destination (string, 255)
- heureDepart (datetime)
- heureArrivee (datetime)
- prix (float)

Symfony génère alors `src/Entity/Vol.php` contenant :

```
namespace App\Entity;

use Doctrine\ORM\Mapping as ORM;

#[ORM\Entity]
class Vol
{
    #[ORM\Id]
    #[ORM\GeneratedValue]
    #[ORM\Column(type: 'integer')]
    private int $id;

    #[ORM\Column(type: 'string', length: 255)]
    private string $destination;

    #[ORM\Column(type: 'datetime')]
    private \DateTimeInterface $heureDepart;

    #[ORM\Column(type: 'datetime')]
    private \DateTimeInterface $heureArrivee;

    #[ORM\Column(type: 'float')]
    private float $prix;
}
```

3. Génération et Application des Migrations

Une fois l'entité créée, nous devons générer une migration pour créer la table correspondante :

- php bin/console make:migration
- php bin/console doctrine:migrations:migrate

Cela crée la table vol dans notre base de données.



4. Manipuler la Base de Données avec Doctrine

Ajouter un Vol en Base de Données

Dans VolController.php, créons une méthode pour ajouter un vol sans passer par un formulaire pour le moment :

```
use Doctrine\ORM\EntityManagerInterface;
use Symfony\Component\HttpFoundation\Response;
use Symfony\Component\Routing\Annotation\Route;
use App\Entity\Vol;

class VolController
{
    #[Route('/vol/ajout', name: 'ajouter_vol')]
    public function ajouterVol(EntityManagerInterface $entityManager): Response
    {
        $vol = new Vol();
        $vol->setDestination('Paris');
        $vol->setHeureDepart(new \DateTime('2024-07-01 10:00:00'));
        $vol->setHeureArrivee(new \DateTime('2024-07-01 12:00:00'));
        $vol->setPrix(150.00);

        $entityManager->persist($vol);
        $entityManager->flush();

        return new Response('Vol ajouté avec succès !');
    }
}
```

Récupérer les Vols

Dans VolController.php, ajoutons une méthode pour récupérer et envoyer à notre vue l'ensemble des vols présent en base de données :

```
use App\Repository\VolRepository;

[...]

#[Route('/vols', name: 'liste_vols')]
public function listeVols(VolRepository $volRepository): Response
{
    $vols = $volRepository->findAll();
    return $this->render('vol/liste.html.twig', ['vols' => $vols]);
}
```



Et dans templates/vol/liste.html.twig, dans la partie block body :

```
{% for vol in vols %}
    <p>{{ vol.destination }} - {{ vol.heureDepart|date('H:i') }} → {{ vol.heureArrivee|date('H:i') }} - {{ vol.prix }}€</p>
{% endfor %}
```

Conclusion

Nous avons configuré Doctrine, créé une entité Vol, appliqué des migrations et manipulé la base de données.

Prochaine étape : [gérer les formulaires et la validation des données !](#)